Node.js — Day 17

Request Body, POST Requests, Data Chunks & JSON.parse()

Learning goal: Understand how a client sends data to a Node.js server using a POST request, how the built-in http module receives the request body in chunks, and how JSON data is converted into a JavaScript object.

> **Day 17 milestone:** You sent a real POST request with a JSON body using curl, received the body in Node.js, and converted the JSON string into a JavaScript object with JSON.parse().

1. What Is a Request Body?

The **request body** is the data/payload sent by the client as part of an HTTP request. It is commonly used when the client needs to send data to the backend.

```
POST /users

{
  "name": "Bhavana",
  "email": "test@example.com"
}
```

The JSON data shown after the request headers is the request body when the request is carrying JSON data.

2. GET vs POST

```
GET /users
→ "Give me this data"

POST /users
→ "Here is some data; create/process something"
```

For example, a signup form can send user information with a POST request:

```
POST /users

{
  "name": "Bhavana",
  "email": "bhavana@example.com"
}
```

A simple mental model: **GET** is commonly used to retrieve data; **POST** is commonly used to send data to the server, often to create/process something.

3. What Is Actually Sent When the Body Is JSON?

A JavaScript object exists inside JavaScript. Before JSON data is sent over HTTP, it is commonly serialized into JSON text.

```
Frontend JavaScript object
        ↓
JSON.stringify()
        ↓
JSON text/string
        ↓
HTTP request body
        ↓
Backend
```

For example:

```
const user = {
  name: "Bhavana",
  email: "test@example.com"
};

const jsonData = JSON.stringify(user);
```

The result is JSON text such as:

```
'{"name":"Bhavana","email":"test@example.com"}'
```

It may look object-like when displayed in browser DevTools, but JSON being transmitted is data represented as text/bytes, not a JavaScript object reference.

4. How Node.js Receives the Request Body

With the built-in Node.js http module, request body data can arrive in multiple chunks. We collect those chunks into a variable.

```
let body = "";

req.on("data", (chunk) => {
  body += chunk;
});

req.on("end", () => {
  console.log(body);
});
```

data fires as chunks arrive. **end** tells us that the request body has finished arriving.

5. Why We Use let Instead of const

Because we append data to **body**, its value changes. Therefore we use **let**.

```
let body = "";

body += chunk;
```

Using **const body =** would prevent reassignment and cause a problem when trying to append chunks.

6. Complete Node.js Example

```
const http = require("http");

const server = http.createServer((req, res) => {

  let body = "";

  req.on("data", (chunk) => {
    body += chunk;
  });

  req.on("end", () => {
    console.log(body);

    res.end("Received!");
  });

});
```

```
server.listen(3000);
```

This server collects the incoming body and prints it after all body chunks have arrived.

7. Testing With curl

Opening **localhost:3000** in a normal browser sends a typical GET request. To test a POST request with a JSON body, curl can act as the client.

```
curl -X POST http://localhost:3000/users \
-H "Content-Type: application/json" \
-d '{"name":"Bhavana","email":"test@example.com"}'
```

You do **not** need to memorize the full curl command. Understand its purpose:

```
curl       → sends an HTTP request
-X POST    → use the POST method
-H         → send a header
-d         → send the request body/data
```

The terminal running Node.js is the **server**. A second terminal running curl acts as the **client**.

8. Two-Terminal Mental Model

Terminal 1

Node.js server

server.listen(3000)

↓ waits for requests

Terminal 2

curl client

POST /users + JSON body

↓

localhost:3000

↓

Node.js receives the body

↓

server responds: Received!

9. The Body Is Initially a JSON String

If the client sends JSON, the collected **body** is JSON text/string.

```
'{"name":"Bhavana","email":"test@example.com"}'
```

That is not yet a JavaScript object that your backend can conveniently access with **body.name**.

10. JSON.parse() — String → JavaScript Object

```
const user = JSON.parse(body);
```

Now the JSON string becomes a JavaScript object:

```
{
  name: "Bhavana",
  email: "test@example.com"
}
```

Then the backend can use the data:

```
console.log(user.name);
console.log(user.email);

// Bhavana
// test@example.com
```

11. Connect Day 15 and Day 17

```
Sending JSON:
JavaScript object
      ↓
JSON.stringify()
      ↓
JSON string
      ↓
HTTP request body

Receiving JSON:
HTTP request body
      ↓
body string
      ↓
JSON.parse()
      ↓
JavaScript object
```

Day 15 taught the two conversion operations. Day 17 uses **JSON.parse()** when the backend receives JSON data. The operations work in opposite directions.

12. Complete POST + JSON Example

```
const http = require("http");

const server = http.createServer((req, res) => {

  let body = "";

  req.on("data", (chunk) => {
    body += chunk;
  });

  req.on("end", () => {

    const user = JSON.parse(body);

    console.log(user.name);
    console.log(user.email);

    res.end("User received!");
  });

});

server.listen(3000);
```

If the client sends:

```
{"name":"Bhavana","email":"test@example.com"}
```

the backend can parse it and work with **user.name** and **user.email**.

13. Important Rules

• **Request body** = data/payload sent by the client in the HTTP request.

• **POST** is commonly used when sending data to the server.

• With Node's built-in **http** module, body data can arrive in chunks.

• **req.on("data")** receives chunks.

• **req.on("end")** indicates the body has finished arriving.

• Use **let** when accumulating chunks into a changing body variable.

• JSON sent over HTTP is represented as JSON text/bytes, not a JavaScript object reference.

• **JSON.parse()** converts JSON text into a JavaScript value/object.

• **curl** can act as a client for testing HTTP requests; its exact command does not need to be memorized.

14. Common Confusion Clarified

“Is the payload in the Network tab the request body?” Yes. When a request carries JSON, the JSON shown as the request payload represents the JSON data in the request body.

“Does the client send a JavaScript object through HTTP?” Not literally. A JavaScript object is serialized to JSON text when JSON is the chosen request format.

“Why parse it again?” The backend parses the received JSON text into a JavaScript object so JavaScript code can conveniently access and manipulate its properties.

> **“Do I need to memorize curl?”** No. Understand that curl is acting as the client and sending a POST request with a body.

15. Practice Completed

Question 1 ■ POST is appropriate for sending signup data to create/process an account.

Question 2 ■ The JSON data in the request is the request body.

Practical test ■ Successfully ran the Node.js server and used a second terminal with curl to send a POST request.

Final question ■ JSON.parse(body) produces a JavaScript object, so **user.name** returns **Bhavana**.

Day 17 practice completed successfully. ■

16. Quick Revision

```
GET
→ commonly retrieve data

POST
→ commonly send data

Request body
→ data sent by the client

data event
→ receives body chunks

end event
→ body finished
```

```
JSON.stringify()
→ object → JSON string

JSON.parse()
→ JSON string → JavaScript object
```

Key Takeaway

A client can send JSON data in the request body of a POST request. Node.js receives that body as incoming data, we collect the chunks, and JSON.parse() converts the received JSON text into a JavaScript object that the backend can work with.